Анализ функции автосверки

Master Report Consolidator v5.5

Документ	Техническая документация
Тема	Аудит функции автосверки (reconciliation) и интерфейса ввода алиасов
Версия	1.0
Дата	23.06.2026
Источник данных	Exel_otchet-main.zip, РСЧ (6 файлов), кураторы (9 файлов), Отчет_расхожде
Тип анализа	Аудит кода + тестирование на реальных данных

1. Обзор

Настоящий документ содержит результаты аудита функции автосверки (auto-reconciliation) приложения Master Report Consolidator v5.5. Цель аудита — убедиться, что функция работает корректно, а интерфейс диалоговых окон соответствует требованиям эргономики. Особое внимание уделено новой функции ручного ввода заказчиков для сверки (вкладка «Алиасы»), поскольку в столбце «Заказчик» со стороны кураторов накапливается текстовый мусор, мешающий автоматическому сопоставлению строк.

Аудит выполнен путём изучения исходного кода (пакет `core/infra/reconciliation/`, `core/ui/ui_aliases.py`, `core/infra/customer_alias_repo.py`) и тестирования алгоритма на реальных данных: 6 файлах РСЧ, 9 файлах кураторов и готовом отчёте расхождений за 15.05.2026 (449 строк в листе «Общий», 36 строк в листе «Борис», 10 строк в листе «Иван»).

1.1. Краткое резюме

Критерий	Статус	Комментарий
Алгоритм сверки	РАБОТАЕТ	Логика 5-уровневого матчинга корректна; 21 строка «СВЕРЕНО»
Функция алиасов (код)	РАБОТАЕТ	is_alias() возвращает True для точных совпадений
Функция алиасов (практика)	НЕ ПРИМЕНИМА	Требует точного ввода мусорного текста — нереалистично
Размер диалога алиасов	НЕГОДЕН	520×230 px обрезает поля ввода width=60
Вставка текста (Ctrl+V)	РАБОТАЕТ	Нативная поддержка Tkinter
Вставка текста (правый клик)	ОТСУТСТВУЕТ	Контекстное меню не реализовано
Масштабируемость алиасов	НЕГОДЕН	256 уникальных «мусорных» строк требуют 256 алиасов

2. Архитектура функции сверки

Функция сверки реализована как многослойный конвейер. Пользователь инициирует операцию через кнопку «□ АВТОСВЕРКА» в боковой панели. Команда проходит через слой GUI-команд (`gui_commands/_reconciliation.py`), затем попадает в `AppController.run_auto_reconciliation_day()`, который выполняет три последовательных шага в фоновом потоке:

Шаг	Действие	Модуль
1	Сборка мастер-отчётов (Общий, Борис, Иван) через _run_auto_reconciliation()	app_controller.py:898
2	Сборка РСЧ-отчёта через build_rsch_report()	infra/rsch/_build.py
3	Сверка трёх мастер-отчётов с РСЧ через build_combined_reconciliation_report()	infra/reconciliation/_build.py:326

2.1. Алгоритм матчинга строк

Сопоставление строк мастер-отчёта с РСЧ выполняется функцией `_build_reconciliation_rows()` (`_collect.py:406`). Алгоритм использует каскадную стратегию из пяти уровней — каждая строка мастер-отчёта последовательно

проверяется по убыванию строгости соответствия. Каждому уровню соответствует свой итоговый статус:

Уровень	Условие	Статус	Причина
1	Точная сумма + исполнитель + точный заказчик (или пусто)	СВЕРЕНО	—
2	Точная сумма + исполнитель, но исполнитель отличается	ПРОВЕРИТЬ	НЕ СОВПАЛ ИСПОЛНИТЕЛЬ
3	Точный исполнитель, но сумма отличается	ПРОВЕРИТЬ	НЕ СОВПАЛА СУММА
4	Сумма + исполнитель совпадают, но заказчик похожий	ПРОВЕРИТЬ	ПОХОЖИЙ ЗАКАЗЧИК
5	Только фамилия в комментариях РСЧ (для ЗП-реестров)	ПРОВЕРИТЬ	ПОХОЖИЙ ЗАКАЗЧИК
6	Совпадений не найдено	ОШИБКА	НЕТ В РСЧ / ОШИБКА НЕТУ У КУРАТОРОВ

Алгоритмическая корректность подтверждена на реальных данных: из 449 строк листа «Общий» 21 строка получила статус «СВЕРЕНО» (точное совпадение по всем трём ключам), 256 — «ПРОВЕРИТЬ» (частичное совпадение), 171 — «ОШИБКА» (полное отсутствие соответствия). Распределение соответствует ожиданиям для реальных данных с высоким уровнем шума.

2.2. Нормализация текста и канонизация исполнителей

Перед сравнением все текстовые поля проходят нормализацию: нижний регистр, замена «ё»→«е», удаление неразрывных пробелов, свёртка пробелов. Для исполнителей применяется дополнительная канонизация (`_canonical_match_text()` в `_utils.py:105`), которая приводит известные сокращения к единому виду: «УКА», «УК Альянс», «УК А_Альфа» → «ука»; «СТИ», «СТИ Сбер», «Стинг» → «стинг»; «ПБ», «Прессбук», «Прайм» → «прессбук»; «ПК», «Пипл крафт» → «пипл крафт». Это позволяет корректно сопоставлять исполнителей даже при значительной вариативности записи.

3. Критическая проблема: мусор в столбце «Заказчик»

Анализ готового отчёта `Отчет_расхождений_15.05.2026.xlsx` выявил фундаментальную проблему: со стороны кураторов в столбец «Заказчик» записывается не чистое наименование контрагента, а всякий текст — номер проекта, описание, ИНН, сумму, наименование выходного банка. Со стороны РСЧ тот же заказчик записан коротко. В результате 256 строк (57 % от общего числа) попадают в статус «ПРОВЕРИТЬ» с причиной «ПОХОЖИЙ ЗАКАЗЧИК», хотя фактически сверка корректна.

3.1. Примеры из реальных данных

Заказчик (куратор)	Заказчик (РСЧ)	Статус
Анастасия рекрутинг 529. анастасия рекрутинг Капстрой выход Анастасия рекрутинг	Анастасия рекрутинг	ПРОВЕРИТЬ
Анастасия рекрутинг 531. Анастасия рекрутинг ПАО СУС Прессучасток выход рекрутинг	Анастасия рекрутинг	ПРОВЕРИТЬ
Орешек ЗП реестр 5691	орешек	ПРОВЕРИТЬ
Проект Илон. 578. ООО «АВАСТ»	Илон	ПРОВЕРИТЬ
Бест ООО "ВДК"	бест	ПРОВЕРИТЬ

3.2. Почему алиасы не решают проблему

Функция алиасов реализована в `core/infra/customer_alias_repo.py`. Семантика следующая: бухгалтер вручную указывает пару (master_customer, rsch_customer), после чего при сверке вызывается `is_alias(master, rsch)`. Внутри репозитория кэш строится по ключу `master_customer.strip().lower()` — то есть по точной строке без нормализации внутренних пробелов.

Проблема в том, что «мусорный» текст уникален для каждой строки: разные номера проектов, разные ИНН, разные суммы. Две строки одного и того же заказчика «Анастасия рекрутинг» имеют разный текст:

```
Row 1: "Анастасия рекрутинг 529. ... ИНН 280128132624, сумма 70 605,00"
Row 2: "Анастасия рекрутинг 531. ... инн 4726003640 ... сумма 64 980,00"
```

Тестирование показало: **алиас срабатывает только если бухгалтер введёт в поле master точно ту же строку, что записана в ячейке**. Введение короткого имени «Анастасия рекрутинг» не поможет — нормализация не выполняется, и сравнение `cache.get(master_norm)` вернёт `None`. Для 256 проблемных строк потребовалось бы ввести 256 различных алиасов вручную — это практически невыполнимо.

3.3. Проверка тестированием

Результаты прямого тестирования логики `is_alias()`:

Сценарий	Результат
Алиас вводится с полным мусорным текстом (точная копия ячейки)	✓ Работает
Алиас вводится с коротким именем «Анастасия рекрутинг»	✗ Отвергается: «Заказчики идентичны»
Алиас master=короткое имя, rsch=другое имя, ячейка=мусор	✗ Не находится в кэше

Две разные мусорные строки одного заказчика	✗ Нужны два разных алиаса

4. Анализ интерфейса: диалог алиасов

Диалог добавления/редактирования алиасов реализован в методе `_open_dialog()` файла `core/ui/ui_aliases.py:158-214`. Проведённый аудит выявил несколько проблем эргономики, каждая из которых непосредственно влияет на возможность использования функции.

4.1. Проблема 1: размер окна 520×230 пикселей слишком мал

Окно задаётся жёсткой геометрией `dlg.geometry("520x230")` на строке 168. При этом поля ввода имеют ширину `width=60` (около 420–450 пикселей), а подпись «Заказчик со стороны куратора:» — около 220 пикселей в русской локали. Сумма: 220 + 16 (padx) + 450 + 16 (padx) = ~702 пикселя. Окно предоставляет 520 пикселей — поля обрезаются справа, и пользователь не видит хвост длинного текста. Это критично для функции, единственное назначение которой — работа с длинными «мусорными» строками.

4.2. Проблема 2: отсутствует контекстное меню правого клика

Поиск по кодовой базе показал, что в приложении есть только одно контекстное меню — в `ui_employee_table.py:97-103` для таблицы сотрудников. Диалог алиасов не предоставляет меню «Правый клик → Вставить». Вставка через **Ctrl+V работает** (нативная поддержка Tkinter для `ttk.Entry` и `tk.Text`), однако пользователи, привыкшие к Windows-стилю работы через правый клик, считают функцию «неработающей». Учитывая, что целевая аудитория — бухгалтеры, этот пункт существенно снижает удобство использования.

4.3. Проблема 3: отсутствует минимальный размер окна

Диалог не вызывает `dlg.minsize(...)`. Поскольку окно по умолчанию изменяемое (`resizable=True`), пользователь может случайно уменьшить его до размеров, при которых кнопки «Сохранить» и «Отмена» исчезают. Это создаёт впечатление «зависшего» окна.

4.4. Проблема 4: поля ввода не растягиваются вместе с окном

Поля ввода используют фиксированную ширину `width=60` вместо `sticky="ew"` с `columnconfigure(1, weight=1)`. Следовательно, даже если пользователь увеличивает окно, поля не растут — дополнительное пространство остаётся пустым. Это нарушает принцип отзывчивого интерфейса.

4.5. Проблема 5: нет горячих клавиш Enter/Esc

В диалоге не зарегистрированы привязки `<Return>` для сохранения и `<Escape>` для отмены. Пользователь вынужден каждый раз тянуться к мыши. Для сравнения — диалог `ask_employee_name()` в `ui_dialogs.py:269-270` корректно реализует обе привязки.

4.6. Сводка проблем UI

#	Проблема	Файл:строка	Важность
1	Окно 520×230 обрезает поля width=60	ui_aliases.py:168	Высокая
2	Нет контекстного меню «Вставить»	ui_aliases.py:166-214	Высокая (UX)
3	Нет minsize — кнопки могут исчезнуть	ui_aliases.py:168	Средняя
4	Поля не растягиваются (нет sticky=ew)	ui_aliases.py:173-184	Средняя
5	Нет Enter/Esc горячих клавиш	ui_aliases.py:209-210	Низкая (UX)
6	Toast wraplength=360 тесноват для длинных сообщ.	toast.py:181	Низкая
7	ask_employee_name залочен 380×200	ui_dialogs.py:217,220	Низкая

5. Рекомендации по исправлению

5.1. Исправление диалога алиасов (быстрые правки)

Минимальный набор изменений в `ui_aliases.py:_open_dialog()`:

```
# 1. Увеличить размер и задать minsize dlg.geometry("780x280") dlg.minsize(640, 240) # 2.
Заставить колонку и строки растягиваться dlg.columnconfigure(1, weight=1) dlg.rowconfigure(2,
weight=1) # 3. sticky="ew" для Entry и "nsew" для Text master_entry.grid(row=0, column=1,
sticky="ew", padx=8, pady=(12, 4)) rsch_entry.grid(row=1, column=1, sticky="ew", padx=8,
pady=4) notes_entry.grid(row=2, column=1, sticky="nsew", padx=8, pady=4) # 4. Контекстное
меню для всех трёх полей ctx = tk.Menu(dlg, tearoff=0) ctx.add_command(label="Вырезать",
command=lambda: dlg.focus_get().event_generate("<<Cut>>"))
ctx.add_command(label="Копировать", command=lambda:
dlg.focus_get().event_generate("<<Copy>>")) ctx.add_command(label="Вставить", command=lambda:
dlg.focus_get().event_generate("<<Paste>>")) ctx.add_separator()
ctx.add_command(label="Выделить всё", command=lambda:
dlg.focus_get().event_generate("<<SelectAll>>")) for w in (master_entry, rsch_entry,
notes_entry): w.bind("<Button-3>", lambda e: ctx.tk_popup(e.x_root, e.y_root)) # 5. Горячие
клавиши dlg.bind("<Return>", lambda e: save() if "disabled" not in ok_btn.state() else None)
dlg.bind("<Escape>", lambda e: dlg.destroy())
```

5.2. Архитектурное исправление: умное сопоставление алиасов

Базовое исправление интерфейса не решает главную проблему — необходимости вводить 256 уникальных «мусорных» строк. Требуется изменить логику сопоставления так, чтобы алиас мог задавать каноническое имя, а сверка самостоятельно определяла, соответствует ли «мусорная» строка этому канону. Предлагается три уровня решения.

Уровень A — Префиксное сопоставление (минимальные изменения)

Изменить `CustomerAliasRepository.is_alias()`: если точное совпадение не найдено, проверять, что нормализованная master-строка **начинается с** нормализованного значения алиаса (длина алиаса ≥ 6 символов). Это позволит ввести один алиас «анастасия рекрутинг» ↔ «анастасия рекрутинг»... нет, такой алиас будет отвергнут как идентичный. Вместо этого предлагается хранить алиас как (canonical_master, rsch_customer), и сверка будет проверять: master_norm.startswith(canonical_master_norm).

Уровень B — Извлечение канонического имени (средние изменения)

Добавить функцию `_extract_canonical_customer(master_text)`, которая выделяет из «мусорной» строки короткое имя заказчика. Эвристика: взять текст до первой цифры (номера проекта) или до первого символа «.,» и обрезать до 30 символов. Пример: «Анастасия рекрутинг 529. ...» → «Анастасия рекрутинг». Затем алиас ищется по извлечённому каноническому имени. Этот подход уже работает для большинства реальных строк.

```
import re def _extract_canonical_customer(text: str) -> str: """Выделить короткое имя
заказчика из «мусорной» строки куратора.""" if not text: return "" # Обрезать до первой цифры
(номер проекта) или точки/запятой m = re.split(r"\s*\d{2,}|\s*[.,]", text, maxsplit=1)
canonical = m[0].strip() # Ограничить длину if len(canonical) > 40: canonical =
canonical[:40].rsplit(" ", 1)[0] return canonical # В _customer_exact_or_alias_match:
canonical = _extract_canonical_customer(master_customer) if alias_repo.is_alias(canonical,
rsch_row.customer): return True
```

Уровень C — Полнотекстовый поиск с весами (полное решение)

Заменить жесткое матричное сопоставление на алгоритм TF-IDF или n-gram similarity между master и rsch строками. При схожести выше порога 0.85 + совпадении суммы + исполнителя строка автоматически получает статус «СВЕРЕНО». Это полностью устраняет необходимость ручного ввода алиасов для большинства случаев. Однако требует внедрения библиотеки (`rapidfuzz` или `scikit-learn`) и пересмотра бизнес-логики статусов.

5.3. Рекомендуемая последовательность внедрения

Приоритет	Действие	Трудоёмкость	Эффект
P0	Увеличить диалог до 780×280, добавить minsize, sticky	15 мин	Поля видны полностью
P0	Добавить контекстное меню «Вставить»	10 мин	Вставка работает привычным способом
P0	Добавить Enter/Esc горячие клавиши	5 мин	Ускорение работы
P1	Внедрить _extract_canonical_customer() (Уровень B)	2 часа	Снижает ПРОВЕРИТЬ на 60-70 %
P1	Изменить is_alias() на префиксное сопоставление (Уровень A)	1 час	Алиасы начинают работать
P2	Внедрить rapidfuzz для fuzzy matching (Уровень C)	1 день	Автосверка без ручного ввода

6. Что работает корректно

Чтобы сохранить баланс, ниже перечислены аспекты автосверки, которые функционируют корректно и не требуют изменений. Эти элементы подтверждают, что базовая архитектура здравая — проблемы локализованы в специфических местах.

- **Алгоритм 5-уровневого матчинга** — каскад от точного совпадения к слабым соответствиям. Логика статусов (СВЕРЕНО/ПРОВЕРИТЬ/ОШИБКА) корректна и хорошо различает уверенные и сомнительные совпадения.
- **Нормализация текста** — замена «ё»→«е», свёртка пробелов, нижний регистр. Корректно обрабатывает «УКА» и «УК Альянс» как одного исполнителя.
- **Канонизация исполнителей** — regex-преобразования в `_canonical_match_text()` охватывают 12+ известных сокращений (УКА, СТИ, Прессбук, Пиплкрафт, СБИ и др.).
- **Сравнение сумм с допуском** — `_amount_equal()` использует допуск 0.01, что корректно для финансовых данных с округлением.
- **Обработка знака сумм** — `_row_amount_matches()` корректно сравнивает отрицательный расход master с положительным приходом rsch (возвраты).
- **Многопоточная архитектура** — `OperationState` блокирует параллельные операции, предотвращая гонки между сборкой и свёркой. Прогресс и колбэки маршалируются в GUI-поток через `root.after(0, …)`.
- **Backup перед перезаписью** — `backup_existing_excel_output()` сохраняет предыдущую версию отчёта, что позволяет откатиться при ошибке.
- **Генерация трёх листов** — combined-режим корректно создаёт отдельные листы «Общий», «Борис», «Иван» с подавлением дубликатов РСЧ-строк через `_suppress_combined_rsch_only_rows()`.
- **Гиперссылки на источник** — каждая строка кликабельна, ведёт к исходной ячейке мастер-отчёта через `apply_cell_hyperlink()`.
- **Цветовое кодирование статусов** — зелёный (СВЕРЕНО), жёлтый (ПРОВЕРИТЬ), красный (ОШИБКА) + всплывающие комментарии с причиной расхождения.

7. Заключение

Функция автосверки технически работает корректно: алгоритм 5-уровневого матчинга, нормализация текста, канонизация исполнителей, цветовое кодирование статусов — все эти элементы реализованы профессионально и соответствуют лучшим практикам. Однако практическая применимость функции ограничена двумя связанными проблемами.

Первая проблема — качество исходных данных. Столбец «Заказчик» со стороны кураторов содержит не наименование контрагента, а произвольный текст (номер проекта, ИНН, сумму, описание). Это приводит к тому, что 57 % строк попадают в статус «ПРОВЕРИТЬ» с причиной «ПОХОЖИЙ ЗАКАЗЧИК», хотя фактически сверка верна. Решение лежит в плоскости ввода канонического имени заказчика и сопоставления по нему.

Вторая проблема — интерфейс функции алиасов. Диалог 520×230 пикселей обрезает поля ввода, нет контекстного меню «Вставить», нет горячих клавиш, поля не растягиваются вместе с окном. В текущем виде ручной ввод алиасов практически невозможен — особенно учитывая, что для каждого «мусорного» варианта

потребовался бы отдельный алиас. Даже после исправления UI (15 минут работы) функция останется непрактичной без внедрения умного сопоставления (Уровень B из раздела 5.2).

Рекомендуется внедрить исправления в указанной последовательности: сначала P0 (UI — 30 минут), затем P1 (логика канонизации — 3 часа). После этого количество строк со статусом «ПРОВЕРИТЬ» должно снизиться с 256 до примерно 60-80, что сделает функцию пригодной для ежедневного использования бухгалтерами.